

COMPUTABILITÀ E COMPLESSITÀ

Computabilità:

Cosa può essere computato?

Cosa non può esserlo? (con qualsiasi macchina, linguaggio, ...)

Dove si trova il confine?

Complessità:

Quali sono le risorse minime necessarie (es. tempo di calcolo e memoria) per la risoluzione di un problema?

DECIDIBILITÀ E INDECIDIBILITÀ

Obiettivo: analizzare i limiti della risoluzione dei problemi mediante algoritmi.

Studieremo: il potere computazionale degli algoritmi nella soluzione dei problemi.

Proveremo che esistono problemi che possono essere risolti mediante algoritmi e altri no.

Problemi di decisione

I problemi di decisione sono problemi che hanno come soluzione una risposta SI o NO.

Esempi:

- ▶ PRIMO: Dato un numero x , x è primo?

Problemi di decisione

I problemi di decisione sono problemi che hanno come soluzione una risposta SI o NO.

Esempi:

- ▶ PRIMO: Dato un numero x , x è primo?
- ▶ CONNESSO: Dato un grafo G , G è connesso?

Problemi di decisione

I problemi di decisione sono problemi che hanno come soluzione una risposta SI o NO.

Esempi:

- ▶ PRIMO: Dato un numero x , x è primo?
- ▶ CONNESSO: Dato un grafo G , G è connesso?
- ▶ ACCETTAZIONE DI UN DFA: Dato un DFA $\mathcal{B}$ e una stringa w , l'automa $\mathcal{B}$ accetta w ?

Rappresenteremo i problemi di decisione mediante linguaggi.

Ricorda: l'input per una MdT è sempre una stringa. Se vogliamo dare in input altri oggetti, questi devono essere codificati come stringhe.

Codifica dei problemi di decisione

Rappresenteremo i problemi di decisione mediante linguaggi.

Es.: il linguaggio che rappresenta il problema “PRIMO” è

$$P = \{\langle x \rangle \mid x \text{ è un numero primo}\}$$

dove $\langle x \rangle$ denota una “ragionevole” codifica di x mediante una stringa su un alfabeto Σ .

Codifica dei problemi di decisione

Rappresenteremo i problemi di decisione mediante linguaggi.

Es.: il linguaggio che rappresenta il problema “PRIMO” è

$$P = \{\langle x \rangle \mid x \text{ è un numero primo}\}$$

dove $\langle x \rangle$ denota una “ragionevole” codifica di x mediante una stringa su un alfabeto Σ .

Ad esempio possiamo prendere $\Sigma = \{0, 1\}$ e $\langle x \rangle$ come la codifica binaria di x .

NOTA: $\langle x \rangle \in P$ **se e solo se** PRIMO ha risposta SI su input x

Es.: il linguaggio che rappresenta il problema “CONNESSO” è

$$A = \{\langle G \rangle \mid G \text{ è un grafo connesso}\}$$

dove $\langle G \rangle$ denota una “ragionevole” codifica di G mediante una stringa su un alfabeto Σ .

Codifica dei problemi di decisione

Es.: il linguaggio che rappresenta il problema “CONNESSO” è

$$A = \{\langle G \rangle \mid G \text{ è un grafo connesso}\}$$

dove $\langle G \rangle$ denota una “ragionevole” codifica di G mediante una stringa su un alfabeto Σ .

Possiamo prendere

$$\Sigma = \{0, 1, \dots, 9, (,), \dots\} \text{ e}$$

$$\langle G \rangle = (\{1, 2, 3\}, \{(1, 2), (2, 3), (3, 1)\})$$

Es. Grafi Connessi

Sia $A = \{\langle G \rangle \mid G \text{ è un grafo connesso}\}$ un linguaggio di stringhe che rappresentano grafi connessi (non orientati)

$\langle G \rangle \in A$ se e solo se G è istanza SI per CONNESSO

Risolvere CONNESSO equivale a decidere il linguaggio A

In questo modo esprimiamo un problema computazionale in termini di riconoscimento di un linguaggio (cioè l'insieme delle codifiche di istanze SI per il problema).

PROBLEMI $\equiv$ LINGUAGGI

Rappresenteremo i problemi di decisione mediante linguaggi.

In questo modo esprimiamo un problema computazionale come un problema di riconoscimento di un linguaggio (insieme delle codifiche di istanze SI per il problema).

OBIETTIVO: presentare un problema irrisolvibile

OBIETTIVO: presentare un problema irrisolvibile

I problemi indecidibili sono esoterici o lontani dai problemi di interesse informatico? NO Esempi di problemi indecidibili:

- ▶ Dato un programma in C, esiste un input che lo manda in loop?
- ▶ Dati due programmi essi forniscono lo stesso output?
- ▶ Dato un programma, questo programma è un virus?

Problemi indecidibili

OBIETTIVO: presentare un problema irrisolvibile

Problema

Input: MdT M , stringa w

Domanda: M si arresta su input w ?

Linguaggio corrispondente

$$HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ è una MdT che si arresta su } w \}$$

Problemi indecidibili

OBIETTIVO: presentare un problema irrisolvibile

Problema

Input: MdT M , stringa w

Domanda: M si arresta su input w ?

Linguaggio corrispondente

$$HALT_{TM} = \{\langle M, w \rangle \mid M \text{ è una MdT che si arresta su } w\}$$

Teorema

Il linguaggio $HALT_{TM}$ non è decidibile.

Cardinalità di insiemi (infiniti).

Diagonalizzazione: metodo introdotto da Cantor.

Cardinalità

Cardinalità di un insieme: la sua taglia

Due insiemi hanno la stessa cardinalità se è possibile stabilire una corrispondenza tra i loro elementi.

Es: $A = \{1, 2, 3\}, B = \{4, 3, 5\} \Rightarrow 1 - 4, 2 - 3, 3 - 5$

Cardinalità

Cardinalità di un insieme: la sua taglia

Due insiemi hanno la stessa cardinalità se è possibile stabilire una corrispondenza tra i loro elementi.

Es: $A = \{1, 2, 3\}$, $B = \{4, 3, 5\}$ $\Rightarrow$ $1 - 4, 2 - 3, 3 - 5$

Cosa possiamo dire per insiemi infiniti?

Paradosso di Hilbert

Nel paese senza confini esiste il più grande di tutti gli alberghi:
l'albergo con infinite stanze. Tuttavia anche gli ospiti sono infiniti,
e il proprietario ha esposto un cartello con la scritta COMPLETO.

Paradosso di Hilbert

Nel paese senza confini esiste il più grande di tutti gli alberghi: l'albergo con infinite stanze. Tuttavia anche gli ospiti sono infiniti, e il proprietario ha esposto un cartello con la scritta COMPLETO. Ad un tratto si presenta un viaggiatore che ha assolutamente bisogno di una camera per la notte. Egli non fa questione di prezzo e infine convince l'albergatore, il quale trova il modo di alloggiarlo. Come fa?

Paradosso di Hilbert

Nel paese senza confini esiste il più grande di tutti gli alberghi: l'albergo con infinite stanze. Tuttavia anche gli ospiti sono infiniti, e il proprietario ha esposto un cartello con la scritta COMPLETO.

Ad un tratto si presenta un viaggiatore che ha assolutamente bisogno di una camera per la notte. Egli non fa questione di prezzo e infine convince l'albergatore, il quale trova il modo di alloggiarlo.

Come fa?

Sposta tutti i clienti nella camera successiva (l'ospite della 1 alla 2, quello della 2 alla 3, etc.); in questo modo, è possibile, essendo l'albergo infinito, sistemare (nella camera 1) il nuovo ospite anche se l'albergo è pieno.

Paradosso di Hilbert-2

Poco dopo arriva una comitiva di **infiniti turisti**, anche in questo caso l'albergatore si lascia convincere, in fondo si tratta di un grosso affare, e trova posto ai **nuovi infiniti ospiti** con la stessa facilità con cui aveva alloggiato l'ospite in più

Come fa, senza ripetere infinite volte il passo visto prima?

Paradosso di Hilbert-2

Poco dopo arriva una comitiva di **infiniti turisti**, anche in questo caso l'albergatore si lascia convincere, in fondo si tratta di un grosso affare, e trova posto ai **nuovi infiniti ospiti** con la stessa facilità con cui aveva alloggiato l'ospite in più

Come fa, senza ripetere infinite volte il passo visto prima?

Sposta ogni ospite nella stanza con numero doppio rispetto a quello attuale (dalla 1 alla 2, dalla 2 alla 4, etc.), lasciando ai nuovi ospiti tutte le camere con i numeri dispari, che sono essi stessi infiniti, risolvendo dunque il problema. Gli ospiti sono tutti dunque sistemati, benchè l'albergo fosse pieno.

Paradosso di Hilbert-3

Ancora piú difficile: ci sono infiniti alberghi con infinite stanze tutti al completo. Tutti gli alberghi chiudono, tranne uno. Tutti gli ospiti vogliono alloggiare nell'unico albergo rimasto aperto.

Come fa (senza ripetere infinite volte il passo visto prima)?

Paradosso di Hilbert-3

Ancora piú difficile: ci sono infiniti alberghi con infinite stanze tutti al completo. Tutti gli alberghi chiudono, tranne uno. Tutti gli ospiti vogliono alloggiare nell'unico albergo rimasto aperto.

Come fa (senza ripetere infinite volte il passo visto prima)?

Assegna ad ogni persona una coppia di numeri (n, m) in cui n indica l'albergo di provenienza, e m la relativa stanza. Gli ospiti sono quindi etichettati in questo modo:

$(1, 1)$	$(1, 2)$	$(1, 3)$	$(1, 4)$	$\dots$
$(2, 1)$	$(2, 2)$	$(2, 3)$	$(2, 4)$	$\dots$
$(3, 1)$	$(3, 2)$	$(3, 3)$	$(3, 4)$	$\dots$
$(4, 1)$	$(4, 2)$	$(4, 3)$	$(4, 4)$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$

A questo punto basta assegnare le nuove stanze agli ospiti secondo un criterio ordinato, ad esempio per diagonali:

$(1, 1) \rightarrow 1$; $(1, 2) \rightarrow 2$; $(2, 1) \rightarrow 3$; $(1, 3) \rightarrow 4$; $(2, 2) \rightarrow 5$; $(3, 1) \rightarrow 6$; $\dots$

Cardinalità

Quanti numeri naturali ci sono? INFINITI!

Quanti numeri naturali pari ci sono? INFINITI!

Quanti numeri naturali dispari ci sono? INFINITI!

Quanti numeri **reali** ci sono? INFINITI!

Cardinalità

Quanti numeri naturali ci sono? INFINITI!

Quanti numeri naturali pari ci sono? INFINITI!

Quanti numeri naturali dispari ci sono? INFINITI!

Quanti numeri **reali** ci sono? INFINITI!

La quantità di numeri reali è la stessa di quella dei numeri naturali?

Come si misura la cardinalità di insiemi infiniti?

Il Metodo della diagonalizzazione

Introdotta da Cantor nel 1873 mentre cercava di determinare come stabilire se, dati due insiemi infiniti, uno è più grande dell'altro.

Il Metodo della diagonalizzazione

Introdotta da Cantor nel 1873 mentre cercava di determinare come stabilire se, dati due insiemi infiniti, uno è più grande dell'altro.

- Cantor osservò che due insiemi finiti hanno la stessa cardinalità se gli elementi dell'uno possono essere messi in corrispondenza uno a uno con quelli dell'altro.

Il Metodo della diagonalizzazione

Introdotta da Cantor nel 1873 mentre cercava di determinare come stabilire se, dati due insiemi infiniti, uno è più grande dell'altro.

- Cantor osservò che due insiemi finiti hanno la stessa cardinalità se gli elementi dell'uno possono essere messi in corrispondenza uno a uno con quelli dell'altro.
- Estese questo concetto agli insiemi infiniti.

Premessa: Funzioni

Definizione

Una funzione $f : X \rightarrow Y$ è una relazione input-output.

*X è l'insieme dei possibili input (**dominio**),*

*Y è l'insieme dei possibili output (**codominio**).*

Per ogni input $x \in X$ esiste un solo output $y = f(x) \in Y$.

Premessa: Funzioni

Definizione

Una funzione $f : X \rightarrow Y$ è una relazione input-output.

*X è l'insieme dei possibili input (**dominio**),*

*Y è l'insieme dei possibili output (**codominio**).*

Per ogni input $x \in X$ esiste un solo output $y = f(x) \in Y$.

Definizione

$f : X \rightarrow Y$ è iniettiva se $\forall x, x' \in X \quad x \neq x' \Rightarrow f(x) \neq f(x')$

Premessa: Funzioni

Definizione

Una funzione $f : X \rightarrow Y$ è una relazione input-output.

*X è l'insieme dei possibili input (**dominio**),*

*Y è l'insieme dei possibili output (**codominio**).*

Per ogni input $x \in X$ esiste un solo output $y = f(x) \in Y$.

Definizione

$f : X \rightarrow Y$ è iniettiva se $\forall x, x' \in X \quad x \neq x' \Rightarrow f(x) \neq f(x')$

Definizione

$f : X \rightarrow Y$ è suriettiva se $\forall y \in Y$ si ha $y = f(x)$ per qualche $x \in X$

Premessa: Funzioni

Definizione

Una funzione $f : X \rightarrow Y$ è una relazione input-output.

*X è l'insieme dei possibili input (**dominio**),*

*Y è l'insieme dei possibili output (**codominio**).*

Per ogni input $x \in X$ esiste un solo output $y = f(x) \in Y$.

Definizione

$f : X \rightarrow Y$ è iniettiva se $\forall x, x' \in X \quad x \neq x' \Rightarrow f(x) \neq f(x')$

Definizione

$f : X \rightarrow Y$ è suriettiva se $\forall y \in Y$ si ha $y = f(x)$ per qualche $x \in X$

Definizione

Una funzione $f : X \rightarrow Y$ è una funzione biettiva di X su Y (o una biezione tra X e Y) se f è iniettiva e suriettiva.

Premessa: Funzioni

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7\}$

$1 \rightarrow 2, 2 \rightarrow 2, 5 \rightarrow 4$ è una funzione. Non è nè iniettiva nè suriettiva.

Premessa: Funzioni

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7\}$

$1 \rightarrow 2, 2 \rightarrow 2, 5 \rightarrow 4$ è una funzione. Non è nè iniettiva nè suriettiva.

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7, 9\}$

$1 \rightarrow 2, 2 \rightarrow 4, 5 \rightarrow 7$ è una funzione iniettiva ma non suriettiva.

Premessa: Funzioni

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7\}$

$1 \rightarrow 2, 2 \rightarrow 2, 5 \rightarrow 4$ è una funzione. Non è nè iniettiva nè suriettiva.

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7, 9\}$

$1 \rightarrow 2, 2 \rightarrow 4, 5 \rightarrow 7$ è una funzione iniettiva ma non suriettiva.

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4\}$

$1 \rightarrow 2, 2 \rightarrow 4, 5 \rightarrow 2$ è una funzione suriettiva ma non iniettiva.

Premessa: Funzioni

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7\}$

$1 \rightarrow 2, 2 \rightarrow 2, 5 \rightarrow 4$ è una funzione. Non è nè iniettiva nè suriettiva.

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7, 9\}$

$1 \rightarrow 2, 2 \rightarrow 4, 5 \rightarrow 7$ è una funzione iniettiva ma non suriettiva.

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4\}$

$1 \rightarrow 2, 2 \rightarrow 4, 5 \rightarrow 2$ è una funzione suriettiva ma non iniettiva.

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7\}$

$1 \rightarrow 2, 2 \rightarrow 4, 5 \rightarrow 7$ è una funzione biettiva.

Premessa: Cardinalità

Definizione

Due insiemi X e Y hanno la stessa cardinalità se esiste una funzione biettiva $f : X \rightarrow Y$ di X su Y .

Premessa: Cardinalità

Definizione

Due insiemi X e Y hanno la stessa cardinalità se esiste una funzione biettiva $f : X \rightarrow Y$ di X su Y .

$$|X| = |Y| \Leftrightarrow \text{esiste una funzione biettiva } f : X \rightarrow Y$$

Premessa: Cardinalità

Definizione

Due insiemi X e Y hanno la stessa cardinalità se esiste una funzione biettiva $f : X \rightarrow Y$ di X su Y .

$$|X| = |Y| \Leftrightarrow \text{esiste una funzione biettiva } f : X \rightarrow Y$$

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7\} \quad 1 \rightarrow 2, 2 \rightarrow 4, 5 \rightarrow 7.$

Premessa: Cardinalità

Definizione

Due insiemi X e Y hanno la stessa cardinalità se esiste una funzione biettiva $f : X \rightarrow Y$ di X su Y .

$$|X| = |Y| \Leftrightarrow \text{esiste una funzione biettiva } f : X \rightarrow Y$$

Esempio $f : \{1, 2, 5\} \rightarrow \{2, 4, 7\} \quad 1 \rightarrow 2, 2 \rightarrow 4, 5 \rightarrow 7.$

Esempio $f : \mathbb{N} \rightarrow \{2n \mid n \in \mathbb{N}\} \quad n \rightarrow 2n$

Insiemi numerabili

Definizione

Un insieme è enumerabile (o numerabile) se ha la stessa cardinalità di un sottoinsieme di $\mathbb{N}$.

Se A è numerabile possiamo “numerare” gli elementi di A e scrivere una lista $(a_1, a_2, \dots)$

cioè per ogni numero naturale i , possiamo specificare l'elemento i -mo della lista.

Es. Per l'insieme dei numeri naturali pari, l'elemento i -mo della lista corrisponde a $2i$.

Insiemi numerabili

Es. L'insieme dei numeri razionali è numerabile

Mostriamo che possiamo formare una lista di tutti i numeri razionali

Formiamo un rettangolo infinito:

$1/1$	$1/2$	$1/3$	$1/4$	...
$2/1$	$2/2$	$2/3$	$2/4$	...
$3/1$	$3/2$	$3/3$	$3/4$	...
$4/1$	$4/2$	$4/3$	$4/4$	...
...	...	...	...	...

Insiemi numerabili

Come formiamo la lista di tutti i numeri razionali?

Se prendiamo prima tutta la prima linea, non arriviamo mai alla seconda!

$1/1$	$1/2$	$1/3$	$1/4$	$\dots$
$2/1$	$2/2$	$2/3$	$2/4$	$\dots$
$3/1$	$3/2$	$3/3$	$3/4$	$\dots$
$4/1$	$4/2$	$4/3$	$4/4$	$\dots$
$\dots$	$\dots$	$\dots$	$\dots$	$\dots$

Insiemi numerabili

Idea: Listiamo per diagonalali: I, II, III, IV, V, VI,...

1/1	1/2	1/3	1/4	...
2/1	2/2	2/3	2/4	...
3/1	3/2	3/3	3/4	...
4/1	4/2	4/3	4/4	...
...	...	...	...	...

1/1, 2/1, 1/2, 3/1, 2/2, 1/3, ...

Nota: dovremmo eliminare i duplicati (ma è solo una questione tecnica)

Insiemi numerabili

Σ^* è **numerabile**..:

listiamo prima la stringa vuota, poi le stringhe (in ordine lessicografico) lunghe 1, poi 2, ...

Esempio: $\Sigma = \{0, 1\}$, $w_0 = \epsilon$, $w_1 = 0$, $w_2 = 1$, $w_3 = 00$, ...

L'insieme

$$\{\langle M \rangle \mid M \text{ è una MdT sull'alfabeto } \Sigma\}$$

è numerabile: è possibile codificare una MdT M con una stringa su un alfabeto Σ .

L'insieme dei numeri reali non è numerabile.

Sia per assurdo $\mathbb{R}$ numerabile, allora possiamo costruire la lista

$$f(1), f(2), f(3), \dots$$

L'insieme dei numeri reali non è numerabile.

Sia per assurdo $\mathbb{R}$ numerabile, allora possiamo costruire la lista

$$f(1), f(2), f(3), \dots$$

Per ogni $i \geq 1$, scriviamo $f(i) = f_0(i), f_1(i)f_2(i)f_3(i) \dots$

Es: se $f(1) = 4, 256 \dots$

allora $f_0(1) = 4, f_1(1) = 2, f_1(1) = 2, f_2(1) = 5, f_3(1) = 6, \dots$

L'insieme dei numeri reali non è numerabile.

Sia per assurdo $\mathbb{R}$ numerabile, allora possiamo costruire la lista

$$f(1), f(2), f(3), \dots$$

Per ogni $i \geq 1$, scriviamo $f(i) = f_0(i), f_1(i)f_2(i)f_3(i) \dots$

Es: se $f(1) = 4,256 \dots$

allora $f_0(1) = 4, f_1(1) = 2, f_1(1) = 2, f_2(1) = 5, f_3(1) = 6, \dots$

Organizziamoli in una matrice:

[illegible]

L'insieme dei numeri reali non è numerabile (cont.)

$i \backslash f(i)$	f_1	f_2	f_3	$\dots$	$\dots$	$\dots$
1	$f_1(1)$	$f_2(1)$	$f_3(1)$	$\dots$	$f_i(1)$	$\dots$
2	$f_1(2)$	$f_2(2)$	$f_3(2)$	$\dots$	$f_i(2)$	$\dots$
3	$f_1(3)$	$f_2(3)$	$f_3(3)$	$\dots$	$f_i(3)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
i	$f_1(i)$	$f_2(i)$	$f_3(i)$	$\dots$	$f_i(i)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Sia $x \in (0, 1)$ il numero $x = 0, x_1 x_2 x_3 \dots x_i \dots$
ottenuto scegliendo $x_i \neq f_i(i)$ per ogni $i \geq 1$

Chiaramente $x \in \mathbb{R}$.

L'insieme dei numeri reali non è numerabile (cont.)

$i \backslash f(i)$	f_1	f_2	f_3	$\dots$	$\dots$	$\dots$
1	$f_1(1)$	$f_2(1)$	$f_3(1)$	$\dots$	$f_i(1)$	$\dots$
2	$f_1(2)$	$f_2(2)$	$f_3(2)$	$\dots$	$f_i(2)$	$\dots$
3	$f_1(3)$	$f_2(3)$	$f_3(3)$	$\dots$	$f_i(3)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
i	$f_1(i)$	$f_2(i)$	$f_3(i)$	$\dots$	$f_i(i)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Sia $x \in (0, 1)$ il numero $x = 0, x_1 x_2 x_3 \dots x_i \dots$ ottenuto scegliendo $x_i \neq f_i(i)$ per ogni $i \geq 1$

Chiaramente $x \in \mathbb{R}$. **Risulta x nella lista?**

L'insieme dei numeri reali non è numerabile (cont.)

$i \backslash f(i)$	f_1	f_2	f_3	$\dots$	$\dots$	$\dots$
1	$f_1(1)$	$f_2(1)$	$f_3(1)$	$\dots$	$f_i(1)$	$\dots$
2	$f_1(2)$	$f_2(2)$	$f_3(2)$	$\dots$	$f_i(2)$	$\dots$
3	$f_1(3)$	$f_2(3)$	$f_3(3)$	$\dots$	$f_i(3)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
i	$f_1(i)$	$f_2(i)$	$f_3(i)$	$\dots$	$f_i(i)$	$\dots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Sia $x \in (0, 1)$ il numero $x = 0, x_1 x_2 x_3 \dots x_i \dots$
ottenuto scegliendo $x_i \neq f_i(i)$ per ogni $i \geq 1$

Chiaramente $x \in \mathbb{R}$. **Risulta x nella lista?**

Se $x = f(j)$ allora il suo j -mo digit soddisfa $x_j = f_j(j)$;

Ma $x_j \neq f_j(j)$ (per def. di x), **contraddizione!**

Quindi $x \in \mathbb{R}$ non può comparire nella lista e $\mathbb{R}$ non numerabile

Il Metodo della diagonalizzazione

$$\{w_1, w_2, \dots\} = \Sigma^*, \{M_1, M_2, \dots\} = \text{MdT su } \Sigma: \text{ numerabili}$$

Il Metodo della diagonalizzazione

$\{w_1, w_2, \dots\} = \Sigma^*$, $\{M_1, M_2, \dots\} = \text{MdT su } \Sigma$: numerabili

	w_1	w_2	w_3	$\dots$	w_i	w_j
M_1	$x_{1,1}$	.	.	.	.	.
M_2	.	$x_{2,2}$	.	.	.	.
.	.	.	$x_{3,3}$	.	.	.
.	.	.	.	$x_{4,4}$	.	.
M_i	.	.	.	.	$x_{i,i}$	$x_{i,j}$
.	.	.	.	.	.	.
.	.	.	.	.	.	.

con $x_{i,j} = 1$ se $w_j \in L(M_i)$, $x = 0$ altrimenti.

Il Metodo della diagonalizzazione

$\{w_1, w_2, \dots\} = \Sigma^*$, $\{M_1, M_2, \dots\} = \text{MdT su } \Sigma$: numerabili

	w_1	w_2	w_3	$\dots$	w_i	w_j
M_1	$x_{1,1}$	.	.	.	.	.
M_2	.	$x_{2,2}$	.	.	.	.
.	.	.	$x_{3,3}$	.	.	.
.	.	.	.	$x_{4,4}$	.	.
M_i	.	.	.	.	$x_{i,i}$	$x_{i,j}$
.	.	.	.	.	.	.
.	.	.	.	.	.	.

con $x_{i,j} = 1$ se $w_j \in L(M_i)$, $x = 0$ altrimenti.

Sia $L = \{w_i \in \Sigma^* \mid w_i \notin L(M_i)\}$

L è il “complemento della diagonale”:

se l'elemento (M_i, w_i) della diagonale è $x_{i,i} = 1$, allora $w_i \notin L$;

se l'elemento (M_i, w_i) della diagonale è $x_{i,i} = 0$, allora $w_i \in L$

Il Metodo della diagonalizzazione

	w_1	w_2	w_3	$\dots$	w_i	w_j
M_1	$x_{1,1}$	.	.	.	.	.
M_2	.	$x_{2,2}$	.	.	.	.
.	.	.	$x_{3,3}$	.	.	.
.	.	.	.	$x_{4,4}$	.	.
M_i	.	.	.	.	$x_{i,i}$	$x_{i,j}$
.	.	.	.	.	.	.
.	.	.	.	.	.	.

con $x_{i,j} = 1$ se $w_j \in L(M_i)$, $x_{i,j} = 0$ altrimenti.

Sia $L = \{w_i \in \Sigma^* \mid w_i \notin L(M_i)\}$

Può L comparire nella lista?

Il Metodo della diagonalizzazione

	w_1	w_2	w_3	$\dots$	w_i	w_j
M_1	$x_{1,1}$	.	.	.	.	.
M_2	.	$x_{2,2}$	.	.	.	.
.	.	.	$x_{3,3}$	.	.	.
.	.	.	.	$x_{4,4}$	.	.
M_i	.	.	.	.	$x_{i,i}$	$x_{i,j}$
.	.	.	.	.	.	.
.	.	.	.	.	.	.

con $x_{i,j} = 1$ se $w_j \in L(M_i)$, $x_{i,j} = 0$ altrimenti.

Sia $L = \{w_i \in \Sigma^* \mid w_i \notin L(M_i)\}$

Può L comparire nella lista?

Supponiamo $L = L(M_h)$,

- $w_h \in L \Rightarrow x_{h,h} = 0 \Rightarrow w_h \notin L(M_h) = L$ contraddizione
- $w_h \notin L \Rightarrow x_{h,h} = 1 \Rightarrow w_h \in L(M_h) = L$ contraddizione

Il Metodo della diagonalizzazione

“Esistono più linguaggi (problemi) che macchine di Turing (algoritmi)”

Il Metodo della diagonalizzazione

“Esistono più linguaggi (problemi) che macchine di Turing (algoritmi)”

Corollary

Esistono linguaggi che non sono Turing riconoscibili

Macchina di Turing Universale

- ▶ Una MdT universale U simula la computazione di una qualsiasi MdT M

Macchina di Turing Universale

- ▶ Una MdT universale U simula la computazione di una qualsiasi MdT M
- ▶ U riceve in input una **rappresentazione** $\langle M, w \rangle$ di M e di un possibile input w di M

Macchina di Turing Universale

Ricorda: Abbiamo visto che è possibile codificare una MdT M e una stringa w con una stringa su un alfabeto Σ .

Es. $\langle M, w \rangle = \text{"codifica di } M" \# \text{"codifica di } w"$.

Macchina di Turing Universale

- ▶ Una MdT universale U simula la computazione di una qualsiasi MdT M
- ▶ U riceve in input una **rappresentazione** $\langle M, w \rangle$ di M e di un possibile input w di M
- ▶ È chiamata universale perchè la computazione di una qualsiasi MdT può essere simulata da U

$$\langle M, w \rangle \rightarrow \boxed{\text{Macchina Universale } U} \rightarrow \begin{cases} \text{accetta} & \text{se } M \text{ accetta } w \\ \text{rifiuta} & \text{se } M \text{ rifiuta } w \end{cases}$$

A_{TM} è Turing riconoscibile

Teorema

Il linguaggio

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ è una MdT che accetta la parola } w \}$$

è Turing riconoscibile.

A_{TM} è Turing riconoscibile

Teorema

Il linguaggio

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ è una MdT che accetta la parola } w \}$$

è Turing riconoscibile.

Dimostrazione.

Definiamo una MdT U che accetta A_{TM} : sull'input $\langle M, w \rangle$ dove M è una MdT e w è una stringa



A_{TM} è Turing riconoscibile

Teorema

Il linguaggio

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ è una MdT che accetta la parola } w \}$$

è Turing riconoscibile.

Dimostrazione.

Definiamo una MdT U che accetta A_{TM} : sull'input $\langle M, w \rangle$ dove M è una MdT e w è una stringa

1. Simula M sull'input w .



A_{TM} è Turing riconoscibile

Teorema

Il linguaggio

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ è una MdT che accetta la parola } w \}$$

è Turing riconoscibile.

Dimostrazione.

Definiamo una MdT U che accetta A_{TM} : sull'input $\langle M, w \rangle$ dove M è una MdT e w è una stringa

1. Simula M sull'input w .
2. Se M accetta, accetta; se M rifiuta, rifiuta.



Dettagli: Simulazione di MdT input

Abbiamo visto MdT che simula Automa

Simulare MdT M con altra MdT risulta molto simile.

Dettagli: Simulazione di MdT input

Abbiamo visto MdT che simula Automa

Simulare MdT M con altra MdT risulta molto simile.

1. Marca stato iniziale di M (stato corrente) e primo simbolo su nastro (posizione corrente testina)
2. cerca prossima transizione (nella parte che descrive la funzione di transizione),
sia $(q, x) \rightarrow (q', x', D)$
3. Esegui la transizione
4. Aggiorna lo stato corrente (marca q') e la posizione corrente della testina (marca simbolo a D)
5. Se lo stato corrente risulta q_{accept}/q_{reject} decidi di conseguenza, altrimenti ripeti da 2

1. **Nota:** U è detta MdT universale.
2. **Nota:** U riconosce A_{TM} : accetta ogni coppia $\langle M, w \rangle \in A_{TM}$

1. **Nota:** U è detta MdT universale.
2. **Nota:** U riconosce A_{TM} : accetta ogni coppia $\langle M, w \rangle \in A_{TM}$
3. **Nota:** U cicla su $\langle M, w \rangle$ se (e solo se) M cicla su w . Quindi U **non decide** A_{TM} .

Indecidibilità del problema della fermata

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ è una MdT e } w \in L(M)\}$$

Teorema

Il linguaggio A_{TM} non è decidibile.

Paradosso di Bertrand Russel

In un paese vive un solo barbiere, un uomo ben sbarbato, che rade tutti e soli gli uomini del villaggio che non si radono da soli.

Chi sbarba il barbiere?

Paradosso di Bertrand Russel

In un paese vive un solo barbiere, un uomo ben sbarbato, che rade tutti e soli gli uomini del villaggio che non si radono da soli.

Chi sbarba il barbiere?

- se il barbiere rade se stesso,

Paradosso di Bertrand Russel

In un paese vive un solo barbiere, un uomo ben sbarbato, che rade tutti e soli gli uomini del villaggio che non si radono da soli.

Chi sbarba il barbiere?

- se il barbiere rade se stesso,
allora per definizione il barbiere non rade se stesso;

Paradosso di Bertrand Russel

In un paese vive un solo barbiere, un uomo ben sbarbato, che rade tutti e soli gli uomini del villaggio che non si radono da soli.

Chi sbarba il barbiere?

- ▶ se il barbiere rade se stesso,
allora per definizione il barbiere non rade se stesso;
- ▶ se il barbiere non rade se stesso

Paradosso di Bertrand Russel

In un paese vive un solo barbiere, un uomo ben sbarbato, che rade tutti e soli gli uomini del villaggio che non si radono da soli.

Chi sbarba il barbiere?

- ▶ se il barbiere rade se stesso, allora per definizione il barbiere non rade se stesso;
- ▶ se il barbiere non rade se stesso allora, dato che il barbiere rade tutti quelli che non si radono da soli, il barbiere rade se stesso.

Paradosso di Bertrand Russel

In un paese vive un solo barbiere, un uomo ben sbarbato, che rade tutti e soli gli uomini del villaggio che non si radono da soli.

Chi sbarba il barbiere?

- ▶ se il barbiere rade se stesso, allora per definizione il barbiere non rade se stesso;
- ▶ se il barbiere non rade se stesso allora, dato che il barbiere rade tutti quelli che non si radono da soli, il barbiere rade se stesso.

Si tratta di un'antinomia: compresenza di due affermazioni contraddittorie che possono essere entrambe dimostrate o giustificate.

In generale Russel pose il problema dell'insieme di tutti gli insiemi che non contengono se stessi.

Autoreferenza può causare problemi!

Indecidibilità del problema della fermata: Dimostrazione

Supponiamo per assurdo che esiste una macchina di Turing H con due possibili risultati di una computazione (accettazione, rifiuto) e tale che:

$$H = \begin{cases} accetta & \text{se } M \text{ accetta } w \\ rifiuta & \text{se } M \text{ non accetta } w \end{cases}$$

Indecidibilità del problema della fermata: Dimostrazione

Supponiamo per assurdo che esiste una macchina di Turing H con due possibili risultati di una computazione (accettazione, rifiuto) e tale che:

$$H = \begin{cases} accetta & \text{se } M \text{ accetta } w \\ rifiuta & \text{se } M \text{ non accetta } w \end{cases}$$

$$H : \langle M, w \rangle \rightarrow \boxed{H} \rightarrow \begin{cases} accetta & \text{se } M \text{ accetta } w \\ rifiuta & \text{se } M \text{ non accetta } w \end{cases}$$

Indecidibilità del problema della fermata: Dimostrazione

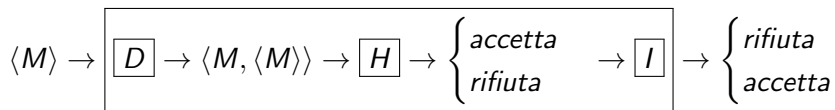
Costruiamo una nuova MdT D che usa H come sottoprogramma
 D sull'input $\langle M \rangle$, dove M è una MdT:

1. Simula H sull'input $\langle M, \langle M \rangle \rangle$
2. Fornisce come output l'opposto di H , cioè se H accetta, *rifiuta* e se H rifiuta, *accetta*

Indecidibilità del problema della fermata: Dimostrazione

Costruiamo una nuova MdT D che usa H come sottoprogramma
 D sull'input $\langle M \rangle$, dove M è una MdT:

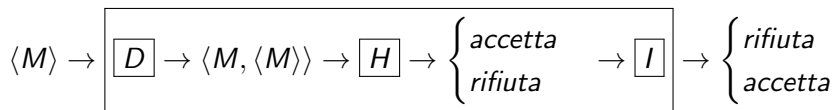
1. Simula H sull'input $\langle M, \langle M \rangle \rangle$
2. Fornisce come output l'opposto di H , cioè se H accetta, *rifiuta* e se H rifiuta, *accetta*



Indecidibilità del problema della fermata: Dimostrazione

Costruiamo una nuova MdT D che usa H come sottoprogramma
 D sull'input $\langle M \rangle$, dove M è una MdT:

1. Simula H sull'input $\langle M, \langle M \rangle \rangle$
2. Fornisce come output l'opposto di H , cioè se H accetta, *rifiuta* e se H rifiuta, *accetta*



Quindi

$$D(\langle M \rangle) = \begin{cases} rifiuta & \text{se } M \text{ accetta } \langle M \rangle, \\ accetta & \text{se } M \text{ non accetta } \langle M \rangle \end{cases}$$

Indecidibilità del problema della fermata: Dimostrazione

1. **Nota:** MdT M deve essere in grado di accettare ogni stringa.
2. **Nota:** La codifica $\langle M \rangle$ di M è una stringa.

Indecidibilità del problema della fermata: Dimostrazione

1. **Nota:** MdT M deve essere in grado di accettare ogni stringa.
2. **Nota:** La codifica $\langle M \rangle$ di M è una stringa.
3. **Nota:** Operare una macchina sulla sua codifica è analogo ad usare un compilatore Pascal per compilarlo (il compilatore Pascal è scritto in Pascal).

Indecidibilità del problema della fermata: Dimostrazione

Autoreferenzialità può essere pericolosa!

Indecidibilità del problema della fermata: Dimostrazione

Autoreferenzialità può essere pericolosa!

Se ora diamo in input a D la sua stessa codifica $\langle D \rangle$ abbiamo

Indecidibilità del problema della fermata: Dimostrazione

Autoreferenzialità può essere pericolosa!

Se ora diamo in input a D la sua stessa codifica $\langle D \rangle$ abbiamo

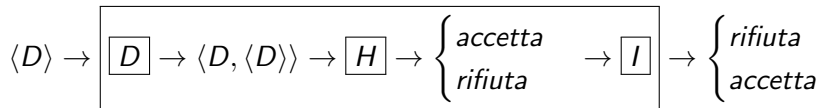
$$D(\langle D \rangle) = \begin{cases} \text{rifiuta} & \text{se } D \text{ accetta } \langle D \rangle, \\ \text{accetta} & \text{se } D \text{ non accetta } \langle D \rangle \end{cases}$$

Indecidibilità del problema della fermata: Dimostrazione

Autoreferenzialità può essere pericolosa!

Se ora diamo in input a D la sua stessa codifica $\langle D \rangle$ abbiamo

$$D(\langle D \rangle) = \begin{cases} rifiuta & \text{se } D \text{ accetta } \langle D \rangle, \\ accetta & \text{se } D \text{ non accetta } \langle D \rangle \end{cases}$$



Indecidibilità del problema della fermata: Dimostrazione

Autoreferenzialità può essere pericolosa!

Se ora diamo in input a D la sua stessa codifica $\langle D \rangle$ abbiamo

$$D(\langle D \rangle) = \begin{cases} \text{rifiuta} & \text{se } D \text{ accetta } \langle D \rangle, \\ \text{accetta} & \text{se } D \text{ non accetta } \langle D \rangle \end{cases}$$

$$\langle D \rangle \rightarrow \boxed{D} \rightarrow \langle D, \langle D \rangle \rangle \rightarrow \boxed{H} \rightarrow \begin{cases} \text{accetta} \\ \text{rifiuta} \end{cases} \rightarrow \boxed{I} \rightarrow \begin{cases} \text{rifiuta} \\ \text{accetta} \end{cases}$$

Cioè D accetta $\langle D \rangle$ se e solo se D non accetta $\langle D \rangle$

Assurdo!

Tutto causato dall'assunzione che esiste H !

Quindi H non esiste!



Indecidibilità del problema della fermata: Riepilogo della Dimostrazione

1. Definiamo $A_{TM} = \{\langle M, w \rangle \mid M \text{ è MdT che accetta } w\}$
2. **Assumiamo A_{TM} decidibile; sia H MdT che lo decide**
3. Usiamo H per costruire MdT D che inverte le decisioni;
 $D(\langle M \rangle)$: accetta se M non accetta $\langle M \rangle$; rifiuta se M accetta $\langle M \rangle$.
4. Diamo in input a D la sua codifica $\langle D \rangle$:
 $D(\langle D \rangle)$: accetta sse D rifiuta.

Contraddizione

Diagonalizzazione?

Consideriamo la tavola

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	acc		acc		...
M_2	acc	acc	acc	acc	...
M_3					
M_2	acc	acc			...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Diagonalizzazione?

Consideriamo H :

MdT H rifiuta anche se M_i va in loop (oltre a se M_i rifiuta)

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...
M_1	acc	rej	acc	rej	...
M_2	acc	acc	acc	acc	...
M_3	rej	rej	rej	rej	...
M_2	acc	acc	rej	rej	...
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Diagonalizzazione?

Consideriamo ora D e $D(\langle D \rangle)$:

Dobbiamo considerare la diagonale!

	$\langle M_1 \rangle$	$\langle M_2 \rangle$	$\langle M_3 \rangle$	$\langle M_4 \rangle$	...	$\langle D \rangle$	...
M_1	acc	rej	acc	rej	...	...	...
M_2	acc	acc	acc	acc	...	...	...
M_3	rej	rej	rej	rej	...	...	...
M_2	acc	acc	rej	rej	...	...	...
:	:	:	:	:	:	:	:
D	acc	acc	rej	rej	...	???	...
:	:	:	:	:	:	:	:
:	:	:	:	:	:	:	:

- ▶ Nella prova precedente è stato utilizzato il metodo della diagonalizzazione.

Indecidibilità

- ▶ Nella prova precedente è stato utilizzato il metodo della diagonalizzazione.
- ▶ In conclusione, A_{TM} è Turing riconoscibile ma è indecidibile.

- ▶ Nella prova precedente è stato utilizzato il metodo della diagonalizzazione.
- ▶ In conclusione, A_{TM} è Turing riconoscibile ma è indecidibile.
- ▶ Che differenza c'è tra le due dimostrazioni?
Cioè che differenza c'è tra U e D ?
- ▶ Sappiamo che esistono linguaggi che non sono Turing riconoscibili.
- ▶ Vogliamo individuare uno specifico linguaggio non Turing riconoscibile ($\overline{A_{TM}}$)

Un linguaggio che non è Turing riconoscibile

Definizione

Diciamo che un linguaggio L è co-Turing riconoscibile se $\bar{L}$ è Turing riconoscibile.

Un linguaggio che non è Turing riconoscibile

Definizione

Diciamo che un linguaggio L è co-Turing riconoscibile se $\bar{L}$ è Turing riconoscibile.

Teorema

Un linguaggio L è decidibile se e solo se L è Turing riconoscibile e co-Turing riconoscibile.

Un linguaggio che non è Turing riconoscibile

L è decidibile $\Leftrightarrow L$ e il suo complemento sono entrambi Turing riconoscibili.

Un linguaggio che non è Turing riconoscibile

L è decidibile $\Leftrightarrow L$ e il suo complemento sono entrambi Turing riconoscibili.

Dimostrazione

($\Rightarrow$) Se L è decidibile allora esiste una macchina di Turing M con due possibili risultati di una computazione (accettazione, rifiuto) e tale che M accetta w se e solo se $w \in L$. Allora L è Turing riconoscibile. Inoltre è facile costruire una MdT $\overline{M}$ che accetta w se e solo se $w \notin L$:

Un linguaggio che non è Turing riconoscibile

L è decidibile $\Leftrightarrow L$ e il suo complemento sono entrambi Turing riconoscibili.

Dimostrazione

($\Rightarrow$) Se L è decidibile allora esiste una macchina di Turing M con due possibili risultati di una computazione (accettazione, rifiuto) e tale che M accetta w se e solo se $w \in L$. Allora L è Turing riconoscibile. Inoltre è facile costruire una MdT $\overline{M}$ che accetta w se e solo se $w \notin L$:

$$w \rightarrow \boxed{M} \rightarrow \begin{cases} \text{accetta} & \text{se } w \in L \\ \text{rifiuta} & \text{se } w \notin L \end{cases} \rightarrow \begin{cases} \text{rifiuta} & \text{se } M \text{ accetta} \\ \text{accetta} & \text{se } M \text{ rifiuta} \end{cases}$$

Un linguaggio che non è Turing riconoscibile

($\Leftarrow$) Supponiamo che L e il suo complemento siano entrambi Turing riconoscibili. Sia M_1 una MdT che riconosce L e M_2 una MdT che riconosce $\bar{L}$. Definiamo una MdT N (a due nastri): sull'input x

Un linguaggio che non è Turing riconoscibile

($\Leftarrow$) Supponiamo che L e il suo complemento siano entrambi Turing riconoscibili. Sia M_1 una MdT che riconosce L e M_2 una MdT che riconosce $\bar{L}$. Definiamo una MdT N (a due nastri): sull'input x

1. Copia x sui nastri di M_1 e M_2

Un linguaggio che non è Turing riconoscibile

($\Leftarrow$) Supponiamo che L e il suo complemento siano entrambi Turing riconoscibili. Sia M_1 una MdT che riconosce L e M_2 una MdT che riconosce $\bar{L}$. Definiamo una MdT N (a due nastri): sull'input x

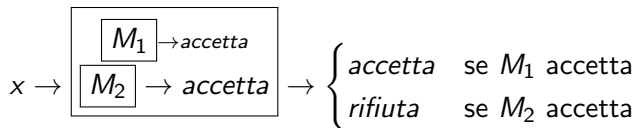
1. Copia x sui nastri di M_1 e M_2
2. Simula M_1 e M_2 in parallelo (usa un nastro per M_1 , l'altro per M_2)

Un linguaggio che non è Turing riconoscibile

($\Leftarrow$) Supponiamo che L e il suo complemento siano entrambi Turing riconoscibili. Sia M_1 una MdT che riconosce L e M_2 una MdT che riconosce $\bar{L}$. Definiamo una MdT N (a due nastri): sull'input x

1. Copia x sui nastri di M_1 e M_2
2. Simula M_1 e M_2 in parallelo (usa un nastro per M_1 , l'altro per M_2)
3. Se M_1 accetta, *accetta*; se M_2 accetta, *rifiuta*

Un linguaggio che non è Turing riconoscibile



Un linguaggio che non è Turing riconoscibile

$$x \rightarrow \boxed{\begin{array}{l} \boxed{M_1} \rightarrow \textit{accetta} \\ \boxed{M_2} \rightarrow \textit{accetta} \end{array}} \rightarrow \begin{cases} \textit{accetta} & \text{se } M_1 \text{ accetta} \\ \textit{rifiuta} & \text{se } M_2 \text{ accetta} \end{cases}$$

N decide L . Infatti, per ogni stringa x abbiamo due casi:

Un linguaggio che non è Turing riconoscibile

$$x \rightarrow \boxed{\begin{array}{l} \boxed{M_1} \rightarrow \text{accetta} \\ \boxed{M_2} \rightarrow \text{accetta} \end{array}} \rightarrow \begin{cases} \text{accetta} & \text{se } M_1 \text{ accetta} \\ \text{rifiuta} & \text{se } M_2 \text{ accetta} \end{cases}$$

N decide L . Infatti, per ogni stringa x abbiamo due casi:

1. $x \in L$. Ma $x \in L$ se e solo se M_1 si arresta e accetta x . Quindi N accetta x .

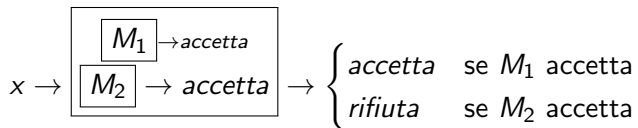
Un linguaggio che non è Turing riconoscibile

$$x \rightarrow \left[\begin{array}{l} M_1 \rightarrow \text{accetta} \\ M_2 \rightarrow \text{accetta} \end{array} \right] \rightarrow \begin{cases} \text{accetta} & \text{se } M_1 \text{ accetta} \\ \text{rifiuta} & \text{se } M_2 \text{ accetta} \end{cases}$$

N decide L . Infatti, per ogni stringa x abbiamo due casi:

1. $x \in L$. Ma $x \in L$ se e solo se M_1 si arresta e accetta x . Quindi N accetta x .
2. $x \notin L$. Ma $x \notin L$ se e solo se M_2 si arresta e accetta x . Quindi N rifiuta x .

Un linguaggio che non è Turing riconoscibile



N decide L . Infatti, per ogni stringa x abbiamo due casi:

1. $x \in L$. Ma $x \in L$ se e solo se M_1 si arresta e accetta x . Quindi N accetta x .
2. $x \notin L$. Ma $x \notin L$ se e solo se M_2 si arresta e accetta x . Quindi N rifiuta x .

Poichè una e solo una delle due MdT

accetta x , N è una MdT

con solo due possibili risultati di una computazione

(accettazione, rifiuto) e tale che N accetta x se e solo se

$x \in L$.



Un linguaggio che non è Turing riconoscibile

Teorema

$\overline{A_{TM}}$ non è Turing riconoscibile.

Dimostrazione.



Un linguaggio che non è Turing riconoscibile

Teorema

$\overline{A_{TM}}$ non è Turing riconoscibile.

Dimostrazione.

Supponiamo per assurdo che $\overline{A_{TM}}$ sia Turing riconoscibile.



Un linguaggio che non è Turing riconoscibile

Teorema

$\overline{A_{TM}}$ non è Turing riconoscibile.

Dimostrazione.

Supponiamo per assurdo che $\overline{A_{TM}}$ sia Turing riconoscibile.
Sappiamo che A_{TM} è Turing riconoscibile.



Un linguaggio che non è Turing riconoscibile

Teorema

$\overline{A_{TM}}$ non è Turing riconoscibile.

Dimostrazione.

Supponiamo per assurdo che $\overline{A_{TM}}$ sia Turing riconoscibile.

Sappiamo che A_{TM} è Turing riconoscibile.

Quindi A_{TM} è Turing riconoscibile e co-Turing riconoscibile.



Un linguaggio che non è Turing riconoscibile

Teorema

$\overline{A_{TM}}$ non è Turing riconoscibile.

Dimostrazione.

Supponiamo per assurdo che $\overline{A_{TM}}$ sia Turing riconoscibile.

Sappiamo che A_{TM} è Turing riconoscibile.

Quindi A_{TM} è Turing riconoscibile e co-Turing riconoscibile.

Per il precedente teorema, A_{TM} è decidibile.



Un linguaggio che non è Turing riconoscibile

Teorema

$\overline{A_{TM}}$ non è Turing riconoscibile.

Dimostrazione.

Supponiamo per assurdo che $\overline{A_{TM}}$ sia Turing riconoscibile.

Sappiamo che A_{TM} è Turing riconoscibile.

Quindi A_{TM} è Turing riconoscibile e co-Turing riconoscibile.

Per il precedente teorema, A_{TM} è decidibile.

Assurdo, poichè abbiamo dimostrato che A_{TM} è indecidibile.

